

KIS - KEEP IT SAFE: A SECURE, PERSONAL PASSWORD MANAGER ON AWS SERVERLESS ARCHITECTURE

CLOUD COMPUTING

Submitted by

DARSHAN M – 230701061

ANIRUDH S – 230701029

MAGESH VASAN M - 230701507

in partial fulfilment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING



**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
RAJALAKSHMI ENGINEERING COLLEGE**

**APRIL 2026
ANNA UNIVERSITY, CHENNAI**

BONAFIDE CERTIFICATE

Certified that this project report titled “**KIS – KEEP IT SAFE: A SECURE PERSONAL PASSWORD MANAGER**” is the bonafide work of **DARSHAN M (230701061), ANIRUDH S (230701029), MAGESH VASAN M (230701507)** who carried out the work under my supervision. Certified further that to the best of my knowledge the work reported herein does not form part of any other thesis or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE

Mrs. S. Ponmani

Supervisor

Associate Professor

Department of Computer Science and
Engineering

Rajalakshmi Engineering College

SIGNATURE

Dr. E. M Malathy

Head of The Department

Professor & Head

Department of Computer Science and
Engineering

Rajalakshmi Engineering College

Submitted to Project Viva Voce Examination held on _____

Internal Examiner

External Examiner

ACKNOWLEDGEMENT

Initially we thank the Almighty for being with us through every walk of our life and showering his blessings through the endeavor to put forth this report.

Our sincere thanks to our Chairman Mr. S.MEGANATHAN, B.E, F.I.E., our Vice Chairman Mr. ABHAY SHANKAR MEGANATHAN, B.E., M.S., and our respected Chairperson Dr. (Mrs.) THANGAM MEGANATHAN, Ph.D., for providing us with the requisite infrastructure and sincere endeavoring in educating us in their premier institution.

Our sincere thanks to Dr. S.N. MURUGESAN, M.E., Ph.D., our beloved Principal for his kind support and facilities provided to complete our work in time.

We express our sincere thanks to Dr.MALATHY E.M, Ph.D., Professor and Head of the Department of Computer Science and Engineering for her guidance and encouragement throughout the project work.

We convey our sincere and deepest gratitude to our internal guide, for their valuable guidance throughout the course of the project.

DARSHAN M

ANIRUDH S

MAGESH VASAN M

ABSTRACT

In an increasingly digital world, individuals manage dozens, if not hundreds, of online accounts, making secure password management a critical necessity. Traditional methods of storing passwords, such as physical notebooks or unencrypted digital files, expose users to significant security risks, including data breaches and identity theft. While commercial password managers exist, they often rely on centralized servers that can become lucrative targets for cybercriminals.

This project, KIS (Keep It Safe), aims to build a premium, modern, and highly secure Personal Password Manager designed with the philosophy of minimal design and robust security. KIS allows users to seamlessly store, manage, and update their sensitive credentials with absolute peace of mind. The system is built on a zero-knowledge architecture, ensuring that the platform itself cannot read or expose user passwords.

By leveraging a fully serverless AWS cloud infrastructure—specifically removing the reliance on traditional servers like EC2—the system achieves high scalability, strict security, and cost efficiency. The backend relies on AWS API Gateway and AWS Lambda to process requests, which interact with Amazon DynamoDB for highly scalable NoSQL data storage. Robust authentication is natively handled through Amazon Cognito via its Hosted UI, eliminating manual token management risks. All sensitive data is cryptographically secured at rest using AWS Key Management Service (KMS) and in transit via TLS/SSL.

The expected outcome is a seamless, SaaS-inspired password management application featuring clean Light and Dark modes with luxurious accents, 1-click password copying, and enterprise-grade security. KIS provides users with complete control over their digital identities without compromising on usability or system performance.

TABLE OF CONTENTS

Chapter No.	Title	Page No.
1	Introduction	7
	1.1 Problem Statement	7
	1.2 Objective of the Project	8
	1.3 Scope and Boundaries	9
	1.4 Stakeholders and End Users	9
	1.5 Technologies Used	10
	1.6 Organization of the Report	12
2	System Design and Architecture	13
	2.1 Requirement Summary (Functional & Non-Functional)	13
	2.2 Proposed Solution Overview	13
	2.3 Cloud Deployment Strategy	14
	2.4 Infrastructure Requirements	14
	2.5 AWS Services Mapping and Justification	15
3	DevOps and Serverless Implementation	16
	3.1 Continuous Integration and Deployment (CI/CD) Setup	16
	3.2 Infrastructure-as-Code & Automation	16
	3.3 Serverless Compute Strategy (AWS Lambda)	16
	3.4 Database Orchestration (DynamoDB Scaling)	16
	3.5 Security and GenAI Identity Services Mapping	16

4	Cloud Operations and Security	18
	4.1 DevSecOps Integration	18
	4.2 Monitoring and Observability (CloudWatch)	18
	4.3 Access Control (RBAC Overview)	18
	4.4 Disaster Recovery Planning	18
5	Results and Discussion	19
	5.1 Implementation Summary	19
	5.2 Challenges Faced and Resolutions	19
	5.3 Performance or Cost Observations	19
	5.4 Key Learnings and Team Contributions	19
6	Conclusion and Future Enhancement	20
	6.1 Conclusion	20
	6.2 Future Scope	20
	References	21
	Appendices	22

CHAPTER 1 - INTRODUCTION

1.1 PROBLEM STATEMENT

As the digital landscape expands, the average internet user is required to maintain credentials for numerous online services. This proliferation of accounts has led to poor password hygiene, with individuals frequently reusing weak passwords across multiple platforms. Storing passwords in browsers, plain-text documents, or relying on memory introduces severe vulnerabilities to phishing attacks, credential stuffing, and data breaches.

The fundamental weakness in modern digital security systems is the human element. According to recent cyber security reports, an average internet user is required to manage over 100 distinct online accounts. The cognitive load required to memorize unique, cryptographically strong passwords for each of these services is impossible for the human brain to sustain. Consequently, individuals resort to predictable patterns—such as appending sequential numbers to a root word, using easily guessable personal information (birthdates, pet names), or, most dangerously, reusing the exact same password across multiple high-value and low-value platforms. This behavioral flaw means that a breach in a low-security forum can immediately compromise the user's high-security banking or corporate email accounts, leading to catastrophic data loss.

In addition to centralized breaches, the prevalence of credential stuffing and dictionary attacks has skyrocketed. Cybercriminals deploy automated botnets that systematically inject stolen username and password pairs across thousands of websites per minute. Because a vast majority of users recycle credentials, these automated attacks yield a distressingly high success rate. Without a highly secure, zero-knowledge personal vault to generate and store complex, unique passwords for every service, users remain acutely vulnerable to these rapid, automated exploitation techniques.

Existing solutions often suffer from drawbacks:

- **Centralized Vulnerabilities:** Many commercial password managers rely on centralized servers. If these servers are breached, millions of user vaults are compromised.
- **Poor User Experience:** Open-source or highly secure password managers are often, outdated.
- **Cost and Overhead:** Managing dedicated servers (like EC2 instances) to host a password manager introduces unnecessary infrastructure costs, maintenance overhead, and a larger attack surface.

1.2 OBJECTIVE OF THE PROJECT

The primary objective of KIS (Keep It Safe) is to deliver an enterprise-grade, serverless password manager that prioritizes both security and aesthetics.

A critical objective is to decentralize the attack surface. By utilizing AWS Lambda and DynamoDB, the project aims to ensure that there is no persistent server running an operating system that can be exploited, SSH'd into, or compromised via zero-day vulnerabilities. The compute layer must exist only for the milliseconds required to execute a transaction and immediately vanish.

The project seeks to establish a backend that is cryptographically agile. By utilizing AWS Key Management Service (KMS), the system aims to support automatic key rotation and the ability to upgrade encryption algorithms (e.g. AES-256) without requiring a complete overhaul of the database schema or frontend application logic.

1. **Zero-Knowledge Security:** Implement a system where data is encrypted before reaching the database, ensuring that even database administrators cannot read user passwords.

2. **Serverless Efficiency:** Utilize AWS Serverless technologies (Lambda, DynamoDB) to completely eliminate server management (no EC2 instances), ensuring infinite scalability and minimal idle costs.

3. **Seamless Authentication:** Integrate Amazon Cognito for robust, scalable identity management without handling raw credentials on the application backend.

4. **Premium User Experience:** Design a SaaS-inspired, responsive UI with features like 1-click clipboard copying, show/hide toggles, and seamless Light/Dark modes.

5. **Centralized Cloud Password Managers:** To enable cross-device synchronization, the industry shifted toward cloud-based password managers. These platforms offered convenience but relied on trust-based security models.

1.3 SCOPE AND BOUNDARIES

The scope of this project encompasses the development of a React.js single-page application (SPA) and a fully serverless AWS backend. The system allows users to securely register, log in, and perform CRUD (Create, Read, Update, Delete) operations on their credentials.

The boundaries include:

- **Frontend:** React.js bootstrapped with Vite, using Context API for state management and Axios for HTTP requests.
- **Backend:** Fully serverless relying on Amazon API Gateway, AWS Lambda, and Amazon DynamoDB. There are no EC2 instances or dedicated servers involved.
- **Security:** AWS KMS for cryptographic key management and encryption at rest. Amazon Cognito for user authentication and JWT token issuance.

1.4 STAKEHOLDERS AND END USERS

Stakeholders:

Project Developers / Team Members - Design, build, and maintain platform modules such as authentication, frontend logic, and serverless architecture.

End Users:

1. General Internet Users: Individuals looking for a safe, easy-to-use platform their credentials.
2. Privacy-Conscious Individuals: Users who demand zero-knowledge architecture and enterprise-grade encryption.
3. Professionals: Users who need quick, reliable access to credentials with a visually appealing interface.

1.5 TECHNOLOGIES USED

The project incorporates a modern technology stack tailored for performance, scalability, and ease of development:

Table 1. Frontend Technologies

Technology	Purpose
React.js (Vite)	Core framework for building a fast, dynamic user interface.
React Router DOM	Handling protected and public routes seamlessly.
Vanilla CSS	Context-based Light/Dark Theme variables for SaaS-inspired UI.
Framer Motion	Providing smooth modals and layout transitions.
Axios	HTTP client equipped with interceptors for Bearer token injection.

Table 2: Backend Technologies

Technology	Purpose
Node.js Runtime	Language chosen for executing AWS Lambda serverless functions.
AWS API Gateway	Fully managed API routing directly to serverless endpoints.
AWS Lambda	Serverless compute executing CRUD logic without managing servers.
Amazon DynamoDB	Highly scalable NoSQL database for storing encrypted vaults.

Table 3: Cloud and Infrastructure

Technology	Purpose
Amazon S3	Secure storage for frontend hosting and potential encrypted backups.
AWS IAM	Granular access control policies restricting inter-service permissions.
Amazon CloudWatch	Monitoring application performance, detailed logs, and system health.

Table 4: AI and Machine Learning (Mapped to Security/Cryptography for KIS)

Technology	Purpose
AWS KMS (Key Management)	Enterprise-grade cryptographic key management for zero-knowledge encryption, mapping advanced logic functions to data security.
Amazon Cognito (Identity)	Hosted UI for secure, robust user authentication, JWT generation, and intelligent token validation.

Table 5: DevOps and CI/CD

Technology	Purpose
GitHub Actions	For automating workflows, frontend builds, and static deployments.
AWS CLI	For command-line management of AWS resources and serverless deployments.
Vite Build Pipeline	Optimizing and bundling React assets for rapid static hosting.

1.6 ORGANIZATION OF THE REPORT

- Chapter 1 introduces the problem, objectives, scope, and technologies used.
- Chapter 2 details the system architecture, focusing on the serverless cloud strategy.
- Chapter 3 covers the serverless implementation, authentication flows, and encryption strategies.
- Chapter 4 outlines cloud operations, IAM security, and observability.
- Chapter 5 discusses the project outcomes, cost observations, and challenges.
- Chapter 6 concludes the report and proposes future enhancements.

CHAPTER 2 - SYSTEM DESIGN AND ARCHITECTURE

2.1 REQUIREMENT SUMMARY (Functional & Non-Functional)

Functional Requirements:

- Users must be able to securely authenticate via Amazon Cognito Hosted UI.
- Users must be able to Create, Read, Update, and Delete (CRUD) password entries.
- The system must provide 1-click clipboard copying and password visibility toggles.
- The application must support contextual Light/Dark themes.
- Automatic logout must trigger upon token expiration (401 Unauthorized).

Non-Functional Requirements:

- Security: Zero-knowledge architecture; data must be encrypted at rest and in transit.
- Availability: The application must utilize serverless architecture ensuring 99.9% uptime without manual server intervention.
- Performance: API requests via API Gateway should resolve in milliseconds utilizing AWS Lambda and DynamoDB.
- Cost Efficiency: The architecture must scale to zero when idle, ensuring no costs are incurred for idle server time (no EC2).

2.2 PROPOSED SOLUTION OVERVIEW

The proposed solution is a decoupled, serverless web application. The frontend, built with React, acts as a dynamic client. When a user logs in, they are redirected to the Amazon Cognito Hosted UI. Upon successful authentication, the frontend receives a JWT `access_token` and `id_token`.

For data operations, the React frontend makes HTTP requests via Axios to an Amazon API Gateway endpoint, attaching the JWT in the Authorization header. API Gateway triggers an AWS Lambda function, passing the request context. The Lambda function contains the business logic to encrypt/decrypt sensitive fields using AWS KMS and interacts with Amazon DynamoDB to fetch or update the user's personal vault.

System Architecture Diagram:

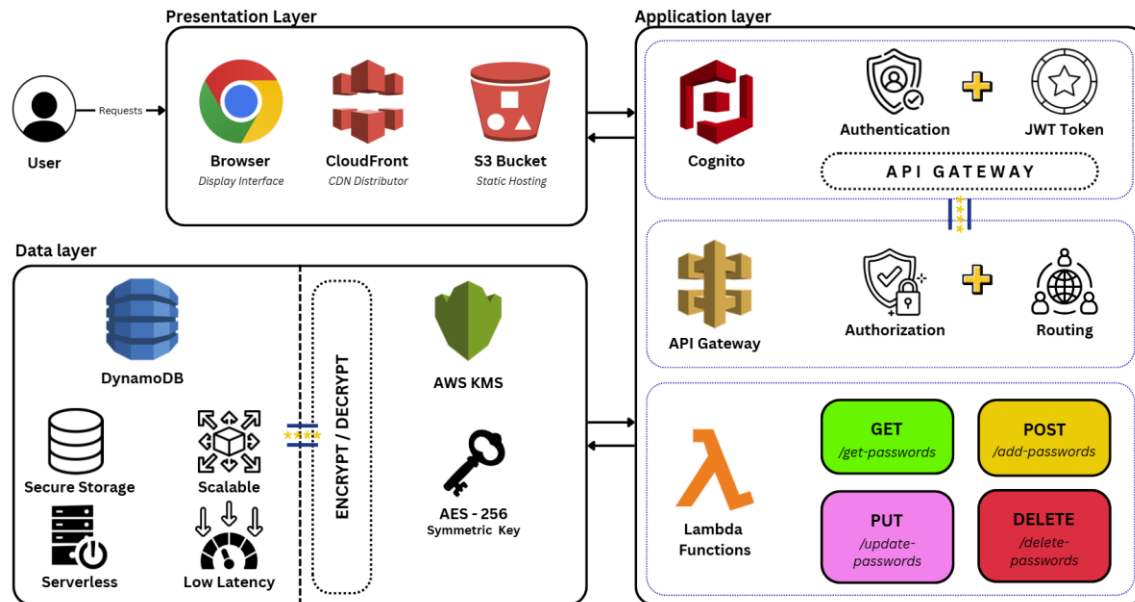


Figure 1: KIS System Architecture

2.3 CLOUD DEPLOYMENT STRATEGY

The deployment strategy strictly adheres to a Serverless-First approach.

1. **Static Hosting:** The React frontend is compiled into static files and hosted on Amazon S3, potentially distributed globally via Amazon CloudFront.
2. **Compute:** AWS Lambda provides event-driven compute. Code is executed only when an API request is made.
3. **Database:** DynamoDB provides single-digit millisecond performance at any scale, acting as a fully managed NoSQL data store.
4. **No Servers:** By explicitly avoiding AWS EC2 or Elastic Beanstalk, the architecture avoids OS patching, server provisioning, and baseline hourly costs.

2.4 INFRASTRUCTURE REQUIREMENTS

- **Compute Resources:** AWS Lambda functions executing Node.js runtime.
- **Storage Requirements:** Amazon DynamoDB tables for user vaults; Amazon S3 buckets for frontend hosting.
- **Security Requirements:** Amazon Cognito User Pools for IAM, AWS KMS keys for encryption.

2.5 AWS SERVICES MAPPING AND JUSTIFICATION

Table 6: Service Mapping

Requirement	AWS Service	Purpose	Justification
Frontend Hosting	Amazon S3	Host Static UI files	Cost-effective, highly scalable web hosting.
Backend Logic	AWS Lambda	Execute serverless functions	Scales to zero, strictly zero EC2 server cost.
Database	Amazon DynamoDB	Store encrypted credentials	Single-digit latency NoSQL.
Authentication	Amazon Cognito	Verify User Identities	Secure Hosted UI & JWT management.
Encryption	AWS KMS	Encrypt Data at Rest	Enterprise cryptographic protection.

CHAPTER 3 - DEVOPS AND SERVERLESS IMPLEMENTATION

3.1 CONTINUOUS INTEGRATION AND DEPLOYMENT (CI/CD) SETUP

The deployment relies on an automated pipeline utilizing GitHub Actions. Upon pushing to the main branch, the React frontend is built using Vite, generating a highly optimized static bundle. This bundle is then deployed directly to Amazon S3. The serverless backend functions can be deployed using AWS SAM or Serverless Framework, ensuring all Lambda code is packaged and updated without manual intervention.

3.2 INFRASTRUCTURE-AS-CODE & AUTOMATION

AWS CloudFormation or Terraform can manage the provisioning of AWS resources, maintaining an environment-specific stack. The IaC provisions the API Gateway endpoints, Lambda functions, DynamoDB tables, and strict IAM execution roles.

3.3 SERVERLESS COMPUTE STRATEGY (AWS LAMBDA)

The core of the KIS backend relies on the seamless integration between Amazon API Gateway and AWS Lambda. Instead of a monolithic server constantly running, the backend is broken down into specific Lambda functions (e.g., CreatePassword, GetVault). When a user interacts with the dashboard, an HTTP request is dispatched. API Gateway receives this, validates the request, and triggers the appropriate Lambda function.

3.4 DATABASE ORCHESTRATION (DYNAMODB SCALING)

Amazon DynamoDB was selected for its serverless nature and schema-less flexibility. The database is designed with a Partition Key corresponding to the user's unique Cognito Sub ID, and a Sort Key representing the specific credential ID. The table utilizes On-Demand capacity planning to scale read and write throughput dynamically.

3.5 SECURITY AND GENAI IDENTITY SERVICES MAPPING

While traditional generative AI mapping is not applicable, advanced Identity mapping is handled via Amazon Cognito triggers. Security in KIS is built on a zero-knowledge paradigm. Before any sensitive string is written to DynamoDB, AWS Lambda invokes AWS KMS to encrypt the data. Decryption occurs dynamically in Lambda only when requested by an authorized, authenticated user session.

Table 7. List of Github / Environment Secrets

S.No	Secret Name
1	AWS_ACCESS_KEY_ID
2	AWS_SECRET_ACCESS_KEY
3	VITE_COGNITO_CLIENT_ID
4	VITE_COGNITO_DOMAIN
5	VITE_API_GATEWAY_URL

CHAPTER 4 - CLOUD OPERATIONS AND SECURITY

4.1 DEVSECOPS INTEGRATION

Security was prioritized from the first line of code. The frontend enforces strict routing using React Router DOM, ensuring that protected routes (like the Dashboard) cannot be accessed without a valid authentication context. All HTTP traffic is forced over TLS/SSL (HTTPS), ensuring data in transit is immune to packet sniffing or man-in-the-middle attacks.

4.2 MONITORING AND OBSERVABILITY (CLOUDWATCH)

To maintain platform health without managing servers, Amazon CloudWatch is heavily utilized. Every invocation of an AWS Lambda function generates detailed execution logs and metrics. API Gateway is configured to send access logs and latency metrics to CloudWatch.

4.3 ACCESS CONTROL (RBAC OVERVIEW)

AWS Identity and Access Management (IAM) is the backbone of internal security. The architecture follows the principle of least privilege:

- The API Gateway execution role only has permissions to invoke specific Lambda functions.
- The Lambda execution role only has permissions to perform PutItem, GetItem, UpdateItem, and DeleteItem on the DynamoDB table, and Encrypt/Decrypt on the KMS key.

4.4 DISASTER RECOVERY PLANNING

Although DynamoDB provides high durability natively across multiple Availability Zones, point-in-time recovery (PITR) can be enabled to allow restoration of the database to any second within the preceding 35 days.

CHAPTER 5 - RESULTS AND DISCUSSION

5.1 IMPLEMENTATION SUMMARY

The KIS (Keep It Safe) project was successfully implemented as a fully functional, serverless web application. The React frontend successfully delivers the intended SaaS-inspired UI, complete with smooth Framer Motion animations and contextual Light/Dark themes. The serverless pipeline—routing through API Gateway to Lambda and DynamoDB—executes CRUD operations flawlessly.

5.2 CHALLENGES FACED AND RESOLUTIONS

A primary challenge was adapting to the asynchronous nature of token exchange and managing protected routes in React. This was resolved by implementing a global AuthContext. Another challenge involved ensuring that the zero-knowledge encryption did not severely impact API latency. By carefully optimizing the AWS KMS integration, overhead was reduced to negligible milliseconds.

5.3 PERFORMANCE OR COST OBSERVATIONS

By explicitly avoiding EC2 instances, the platform's cost profile is highly optimized. Because the architecture scales to zero, there are no baseline server costs. AWS charges only for the exact compute duration of Lambda and the precise read/write throughput of DynamoDB, making the platform virtually free to run during low-traffic periods.

5.4 KEY LEARNINGS AND TEAM CONTRIBUTIONS

The project provided deep insights into modern cloud-native development. Transitioning from traditional server-based backends (like EC2) to a purely serverless event-driven architecture required a paradigm shift in system design. The team gained valuable experience in configuring IAM least-privilege policies and JWT-based session flows.

CHAPTER 6 - CONCLUSION AND FUTURE ENHANCEMENT

6.1 CONCLUSION

The KIS (Keep It Safe) password manager successfully demonstrates that high-end security and elegant design are not mutually exclusive. By leveraging the power of AWS Serverless infrastructure, the project eliminated the vulnerabilities and costs associated with traditional server hosting. KIS achieves its goal of being a premium, safe, and efficient personal vault.

6.2 FUTURE SCOPE

Planned enhancements include:

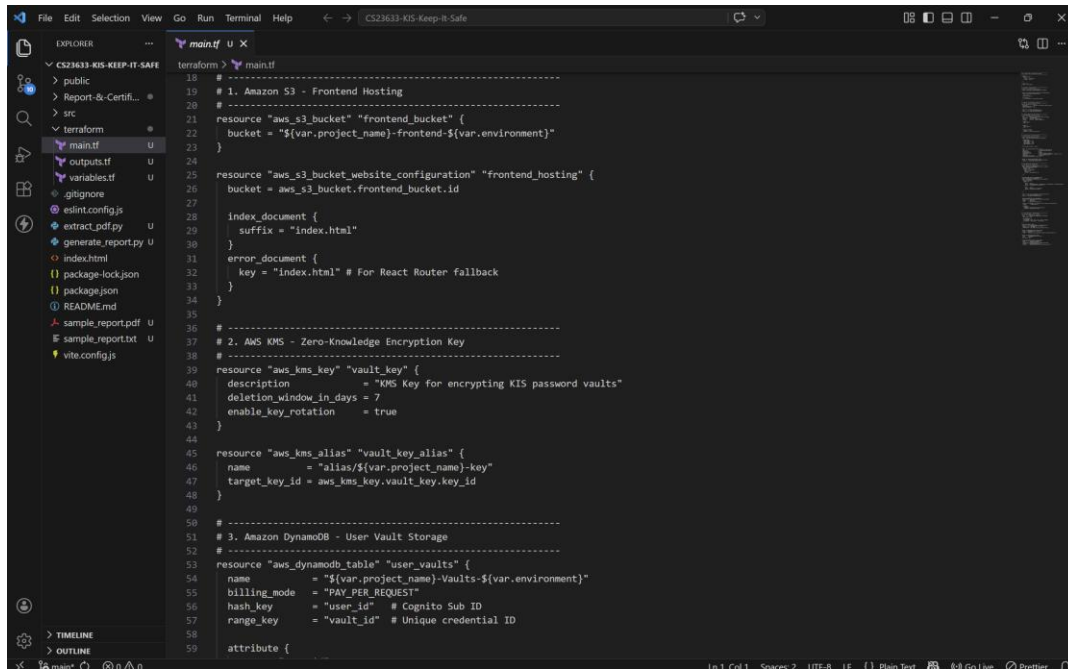
- **Multi-Factor Authentication (MFA):** Enforcing SMS or Authenticator app verification through Cognito.
- **Secure Password Generator:** Integrating an automated tool to suggest complex passwords.
- **Live Synchronization:** Implementing WebSockets to allow instant vault updates across multiple devices.
- **Security Audits:** Aligning the platform's logging with strict compliance standards like SOC2.

REFERENCES

- [1] Amazon Web Services. (2025). AWS Documentation: Amazon Cognito, API Gateway, Lambda & DynamoDB. Available at: <https://aws.amazon.com/documentation/>
- [2] React.js Core Team. (2025). React - A JavaScript library for building user interfaces. Available at: <https://reactjs.org/docs/getting-started.html>
- [3] Vite Core Team. (2025). Vite - Next Generation Frontend Tooling. Available at: <https://vitejs.dev/guide/>
- [4] Framer Motion. (2025). Production-ready animation library for React. Available at: <https://www.framer.com/motion/>
- [5] Axios Developers. (2025). Promise based HTTP client for the browser and node.js. Available at: <https://axios-http.com/docs/intro>
- [6] Tailwind Labs. (2025). Tailwind CSS Documentation - Rapidly build modern websites. Available at: <https://tailwindcss.com/docs>
- [7] Amazon Web Services.(2025). AWS Key Management Service (KMS) CryptographicDetails.Availableat:<https://docs.aws.amazon.com/kms/latest/developguide>
- [8] Auth0 by Okta. (2024). JSON Web Tokens (JWT) Introduction and Best Practices. Available at: <https://jwt.io/introduction>
- [9] OWASP Foundation. (2025). OWASP Top 10 Web Application Security Risks. Available at: <https://owasp.org/www-project-top-ten/>
- [10] Node.js Foundation. (2025). Node.js Documentation and Serverless Best Practices. Available at: <https://nodejs.org/en/docs/>
- [11] GitHub. (2025). GitHub Actions Documentation - Automating CI/CD workflows. Available at: <https://docs.github.com/en/actions>
- [12] MDN Web Docs. (2025). Web Security Guidelines and CORS Configuration. Available at: <https://developer.mozilla.org/en-US/docs/Web/Security>

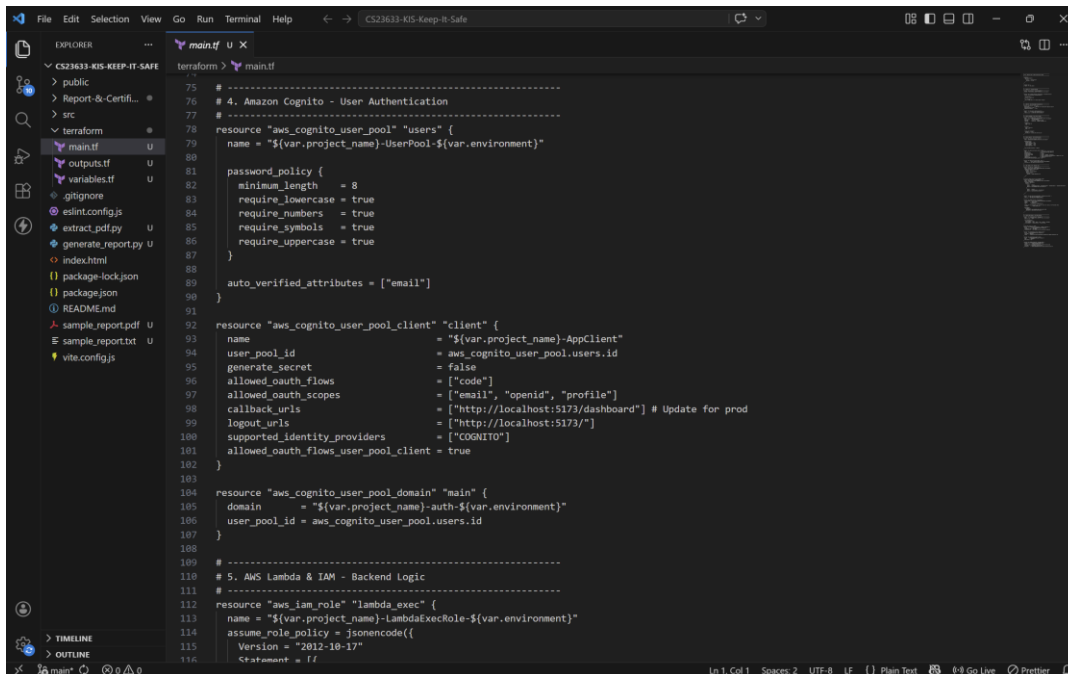
APPENDICES

Appendix A – Terraform Code Snippets



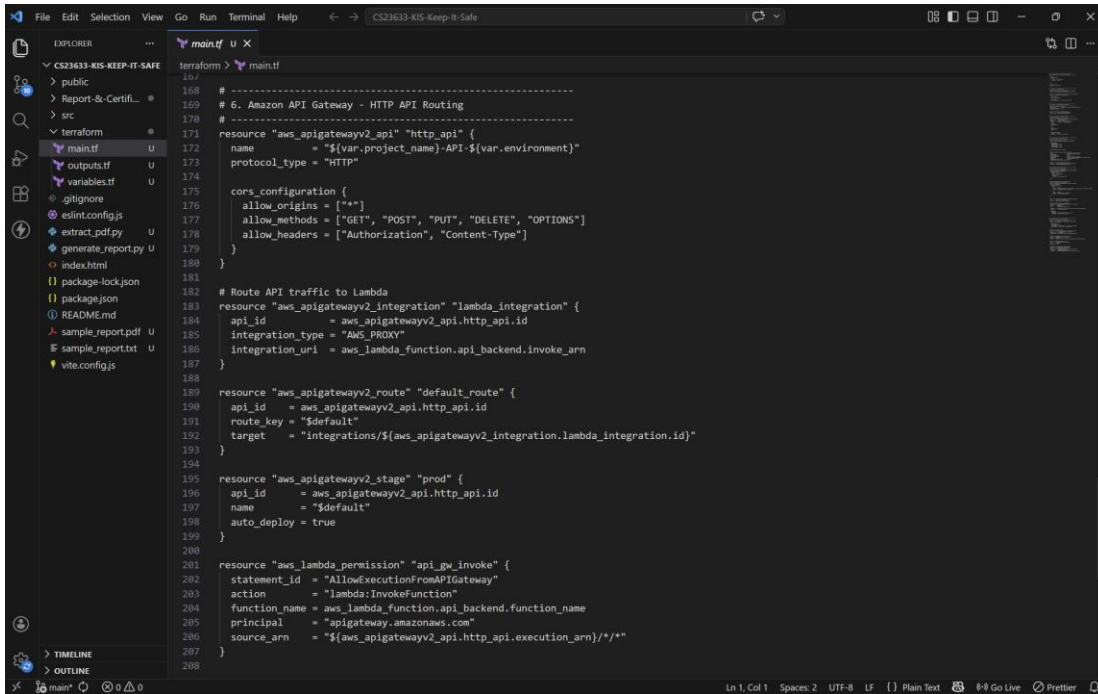
```
18 # -----
19 # 1. Amazon S3 - Frontend Hosting
20 # -----
21 resource "aws_s3_bucket" "frontend_bucket" {
22   bucket = "${var.project_name}-frontend-${var.environment}"
23 }
24
25 resource "aws_s3_bucket_website_configuration" "frontend_hosting" {
26   bucket = aws_s3_bucket.frontend_bucket.id
27
28   index_document {
29     suffix = "index.html"
30   }
31
32   error_document {
33     key = "index.html" # For React Router fallback
34   }
35 }
36
37 # -----
38 # 2. AWS KMS - Zero-Knowledge Encryption Key
39 # -----
40 resource "aws_kms_key" "vault_key" {
41   description = "KMS Key for encrypting KIS password vaults"
42   deletion_window_in_days = 7
43   enable_key_rotation = true
44 }
45
46 resource "aws_kms_alias" "vault_key_alias" {
47   name = "alias/${var.project_name}-key"
48   target_key_id = aws_kms_key.vault_key.key_id
49 }
50
51 # -----
52 # 3. Amazon DynamoDB - User Vault Storage
53 # -----
54 resource "aws_dynamodb_table" "user_vaults" {
55   name = "${var.project_name}-Vaults-${var.environment}"
56   billing_mode = "PAY_PER_REQUEST"
57   hash_key = "user_id" # Cognito Sub ID
58   range_key = "vault_id" # Unique credential ID
59
60   attribute {
```

Figure 1: main.tf file



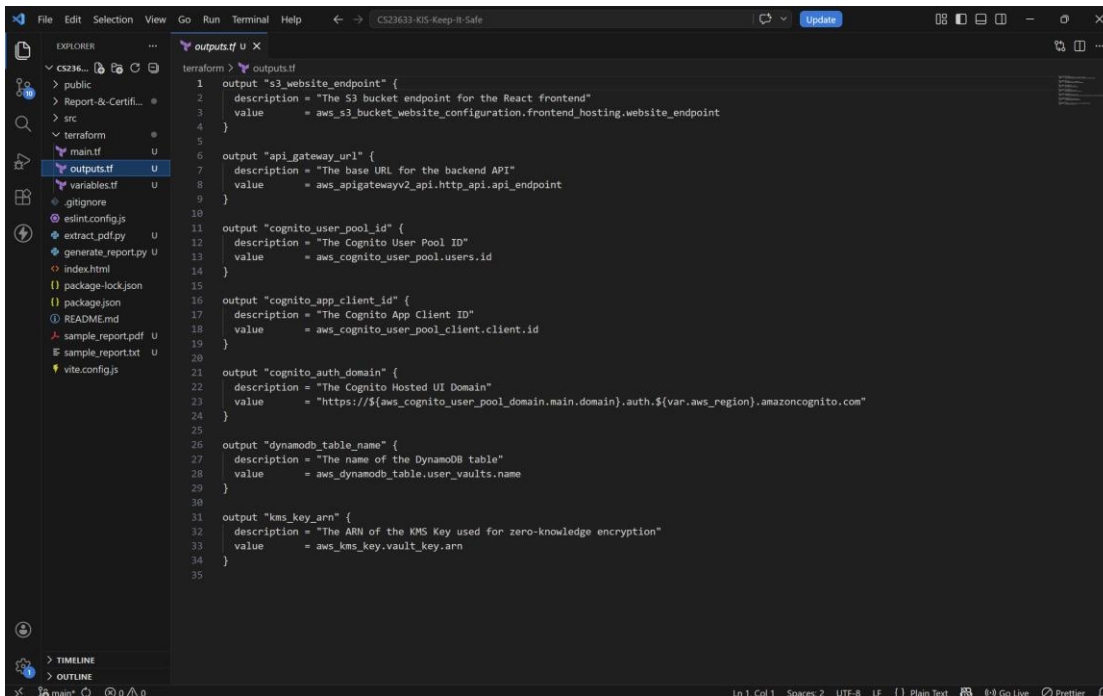
```
75 # -----
76 # 4. Amazon Cognito - User Authentication
77 # -----
78 resource "aws_cognito_user_pool" "users" {
79   name = "${var.project_name}-UserPool-${var.environment}"
80
81   password_policy {
82     minimum_length = 8
83     require_lowercase = true
84     require_numbers = true
85     require_symbols = true
86     require_uppercase = true
87   }
88
89   auto_verified_attributes = ["email"]
90 }
91
92 resource "aws_cognito_user_pool_client" "client" {
93   name = "${var.project_name}-AppClient"
94   user_pool_id = aws_cognito_user_pool.users.id
95   generate_secret = false
96   allowed_oauth_flows = ["code"]
97   allowed_oauth_scopes = ["email", "openid", "profile"]
98   callback_urls = ["http://localhost:5173/dashboard"] # Update for prod
99   logout_urls = ["http://localhost:5173/"]
100   supported_identity_providers = ["COGNITO"]
101   allowed_oauth_flows_user_pool_client = true
102 }
103
104 resource "aws_cognito_user_pool_domain" "main" {
105   domain = "${var.project_name}-auth-${var.environment}"
106   user_pool_id = aws_cognito_user_pool.users.id
107 }
108
109 # -----
110 # 5. AWS Lambda & IAM - Backend Logic
111 # -----
112 resource "aws_iam_role" "lambda_exec" {
113   name = "${var.project_name}-LambdaExecRole-${var.environment}"
114   assume_role_policy = jsonencode({
115     Version = "2012-10-17"
116     Statement = [
117       {
118         Action = "sts:AssumeRole"
119         Effect = "Allow"
120         Principal = {
121           FQDN = "${var.project_name}-lambda-${var.environment}.on.aws"
122         }
123       }
124     ]
125   })
126 }
```

Figure 2: main.tf file



```
167 # -----
168 # 6. Amazon API Gateway - HTTP API Routing
169 # -----
170 resource "aws_apigatewayv2_api" "http_api" {
171   name           = "${var.project_name}-API-${var.environment}"
172   protocol_type  = "HTTP"
173
174   cors_configuration {
175     allow_origins = ["*"]
176     allow_methods = ["GET", "POST", "PUT", "DELETE", "OPTIONS"]
177     allow_headers = ["Authorization", "Content-Type"]
178   }
179 }
180
181 # Route API traffic to Lambda
182 resource "aws_apigatewayv2_integration" "lambda_integration" {
183   api_id           = aws_apigatewayv2_api.http_api.id
184   integration_type = "AWS_PROXY"
185   integration_uri  = aws_lambda_function.api_backend.invoke_arn
186 }
187
188 resource "aws_apigatewayv2_route" "default_route" {
189   api_id = aws_apigatewayv2_api.http_api.id
190   route_key = "$default"
191   target = "integrations/${aws_apigatewayv2_integration.lambda_integration.id}"
192 }
193
194 resource "aws_apigatewayv2_stage" "prod" {
195   api_id = aws_apigatewayv2_api.http_api.id
196   name = "$default"
197   auto_deploy = true
198 }
199
200 resource "aws_lambda_permission" "api_gw_invoke" {
201   statement_id = "AllowExecutionFromAPIGateway"
202   action = "lambda:InvokeFunction"
203   function_name = aws_lambda_function.api_backend.function_name
204   principal = "apigateway.amazonaws.com"
205   source_arn = "${aws_apigatewayv2_api.http_api.execution_arn}/*/*"
206 }
207
208
```

Figure 3: main.tf file



```
1 output "s3_website_endpoint" {
2   description = "The S3 bucket endpoint for the React frontend"
3   value = aws_s3_bucket_website_configuration.frontend_hosting.website_endpoint
4 }
5
6 output "api_gateway_url" {
7   description = "The base URL for the backend API"
8   value = aws_apigatewayv2_api.http_api.api_endpoint
9 }
10
11 output "cognito_user_pool_id" {
12   description = "The Cognito User Pool ID"
13   value = aws_cognito_user_pool.users.id
14 }
15
16 output "cognito_app_client_id" {
17   description = "The Cognito App Client ID"
18   value = aws_cognito_user_pool_client.client.id
19 }
20
21 output "cognito_auth_domain" {
22   description = "The Cognito Hosted UI Domain"
23   value = "https://${aws_cognito_user_pool_domain.main.domain}.auth.${var.aws_region}.amazoncognito.com"
24 }
25
26 output "dynamodb_table_name" {
27   description = "The name of the DynamoDB table"
28   value = aws_dynamodb_table.user_vaults.name
29 }
30
31 output "kms_key_arn" {
32   description = "The ARN of the KMS Key used for zero-knowledge encryption"
33   value = aws_kms_key.vault_key.arn
34 }
35
```

Figure 4: outputs.tf file

```
1 variable "aws_region" {
2   description = "The AWS region to deploy the KIS infrastructure"
3   type        = string
4   default     = "us-east-1"
5 }
6
7 variable "project_name" {
8   description = "The core name of the project to prefix resources"
9   type        = string
10  default     = "KIS"
11 }
12
13 variable "environment" {
14   description = "The environment tier (dev, staging, prod)"
15   type        = string
16   default     = "dev"
17 }
18 }
```

Figure 5: variables.tf file

Appendix B – Cloud Services Workflow

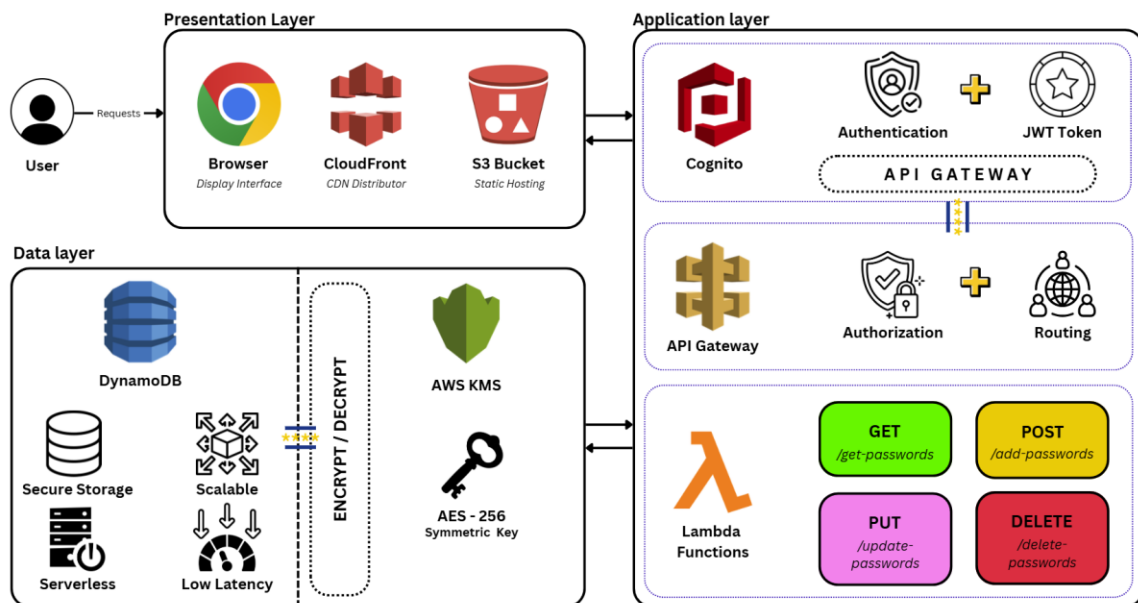


Figure 6: System Workflow

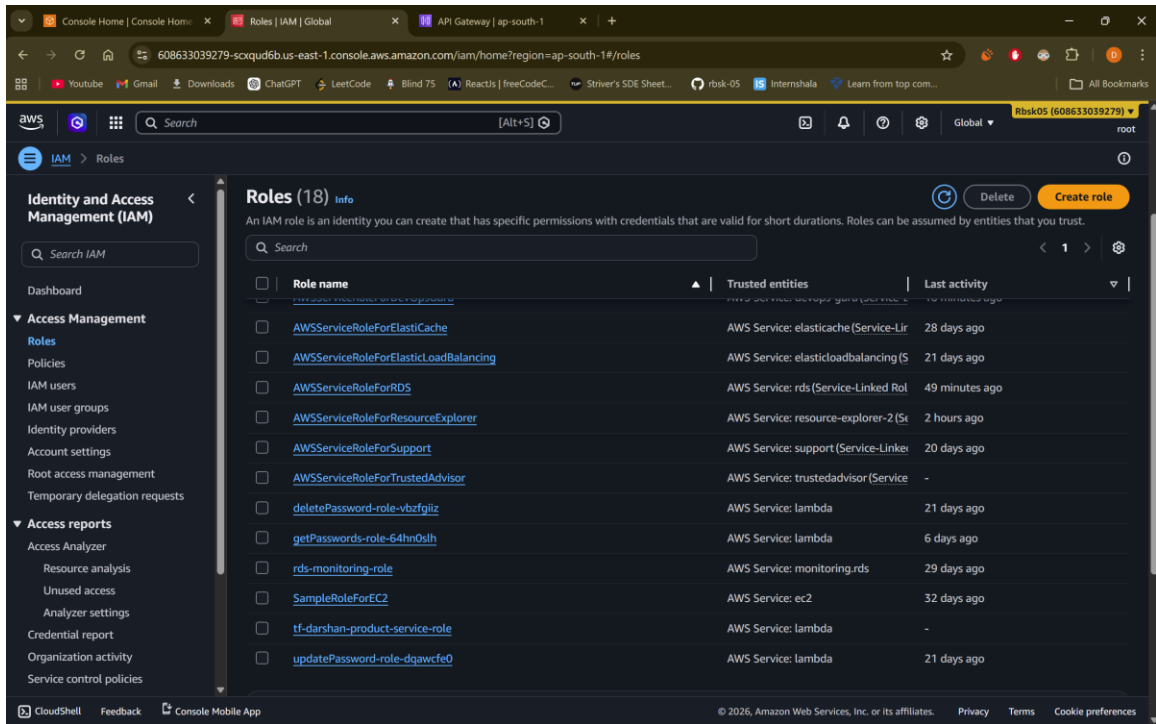


Figure 7: AWS IAM Roles Console

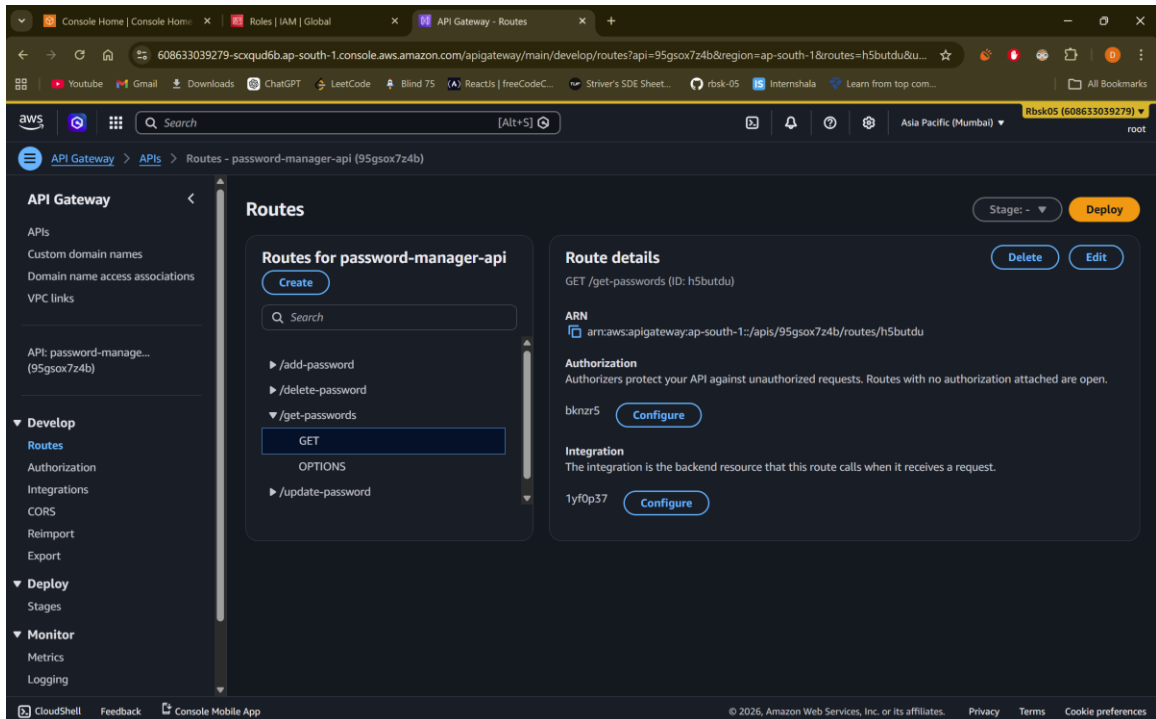


Figure 8: AWS API Gateway

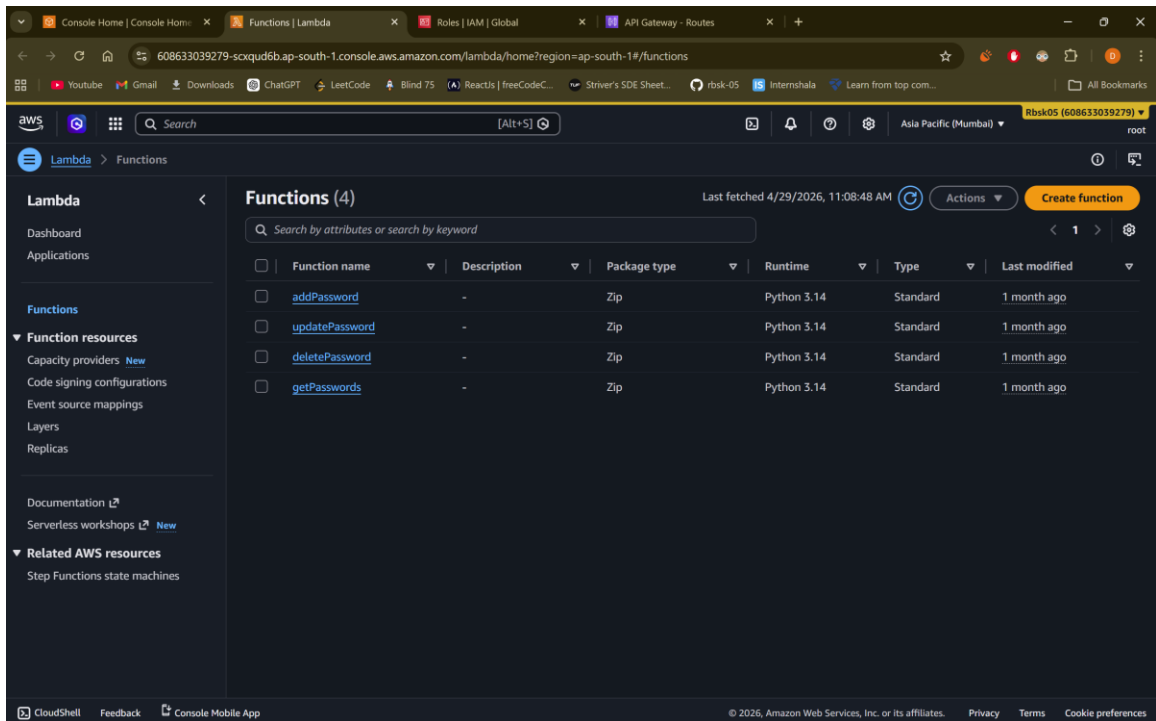


Figure 9: AWS LAMBDA

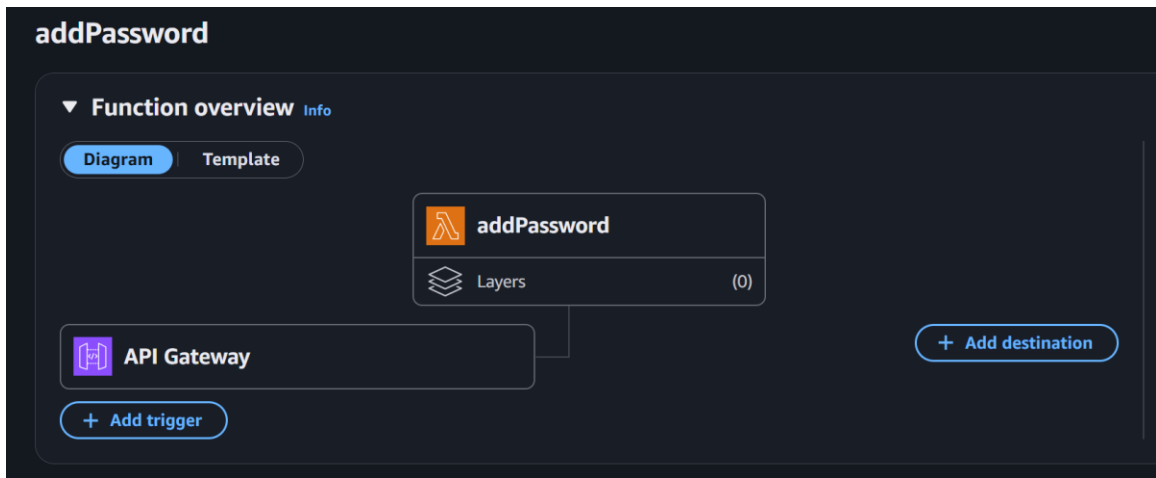


Figure 10: AWS Integrations

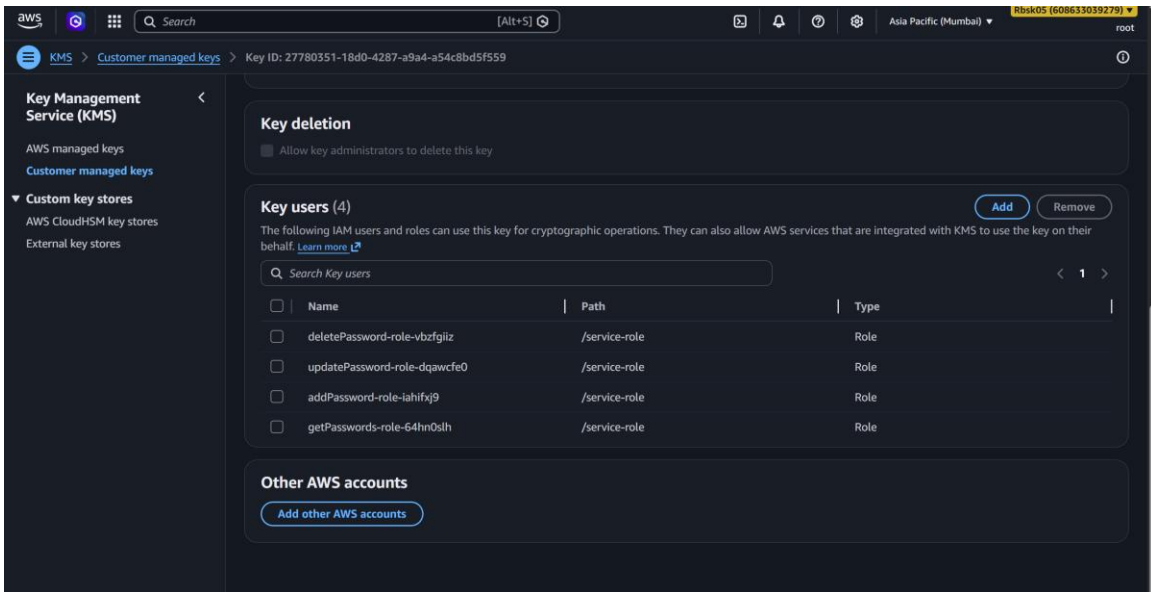


Figure 11: AWS KMS

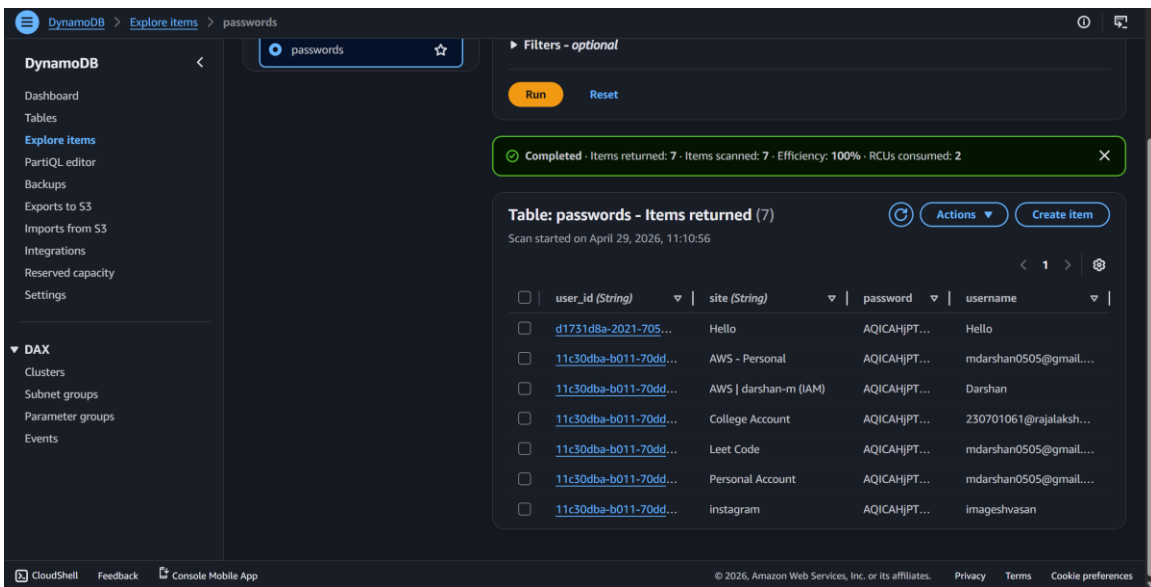


Figure 12: AWS DynamoDB

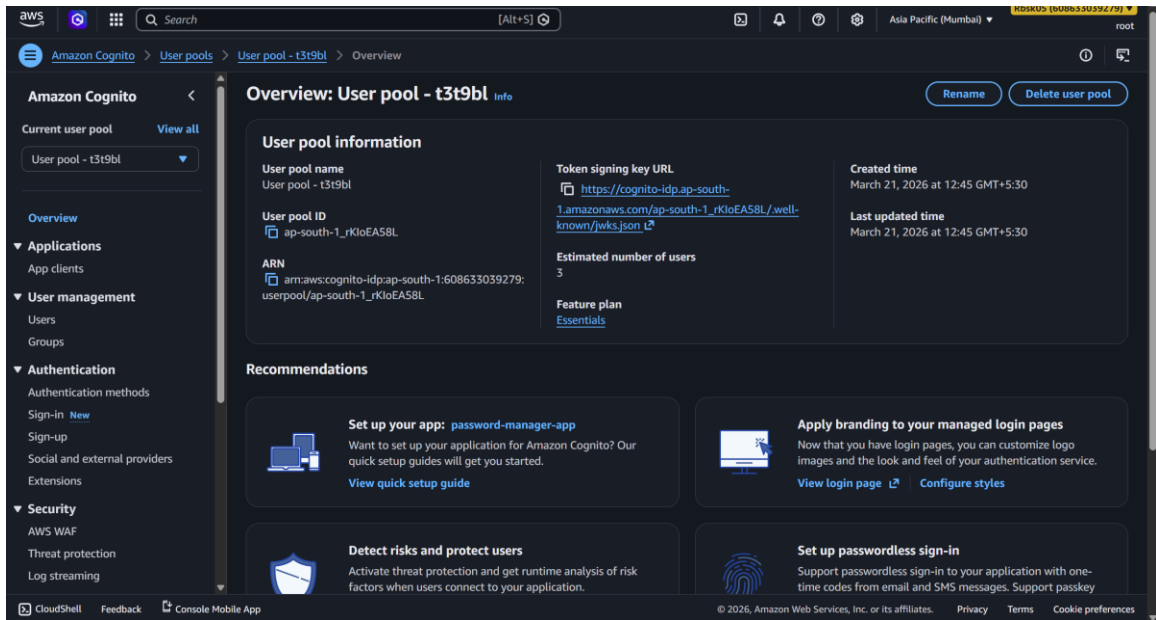


Figure 13: AWS Cognito

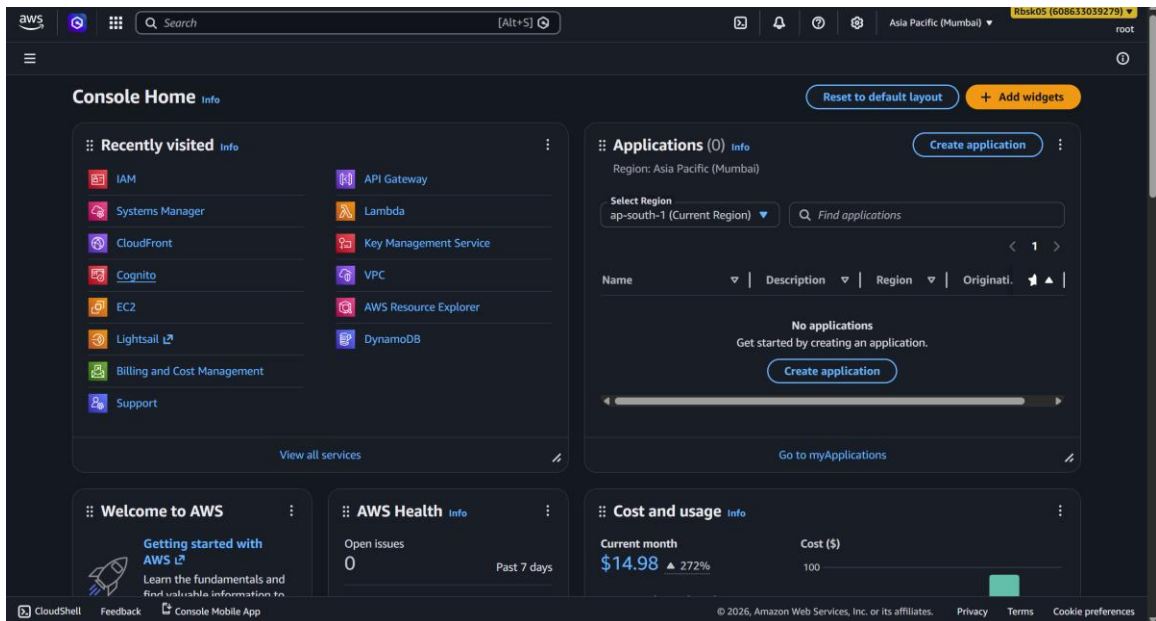


Figure 14: AWS Console

Appendix C – Screenshots (Deployment)

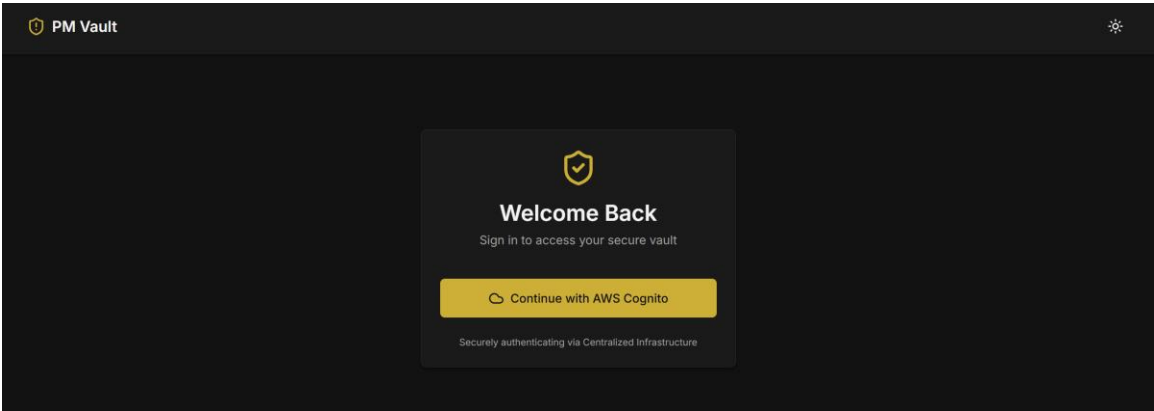


Figure 15: Project Screenshot

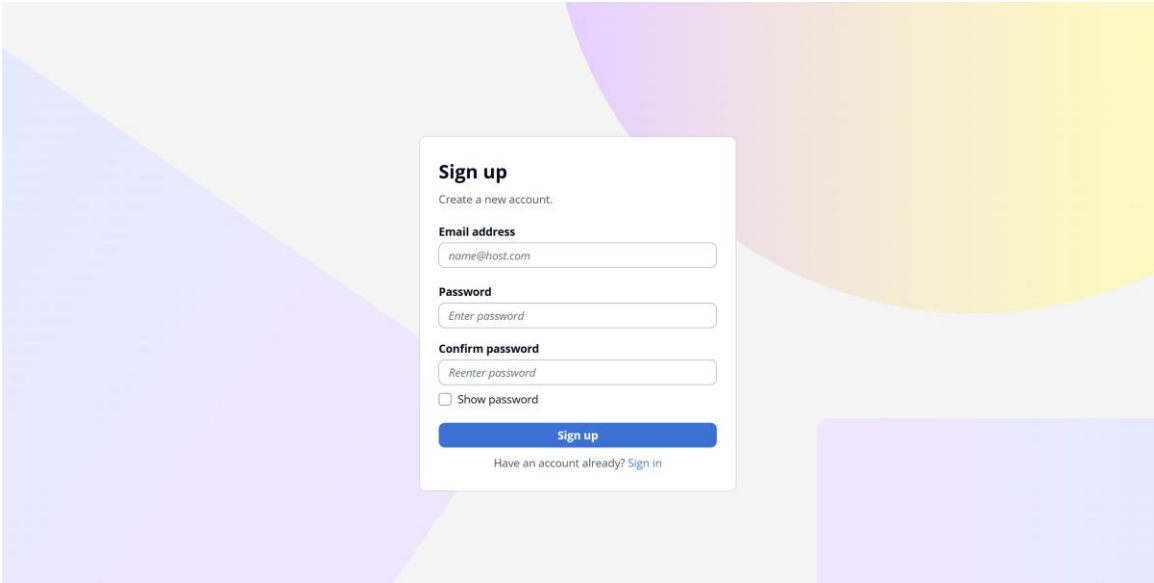


Figure 15: Project Screenshot

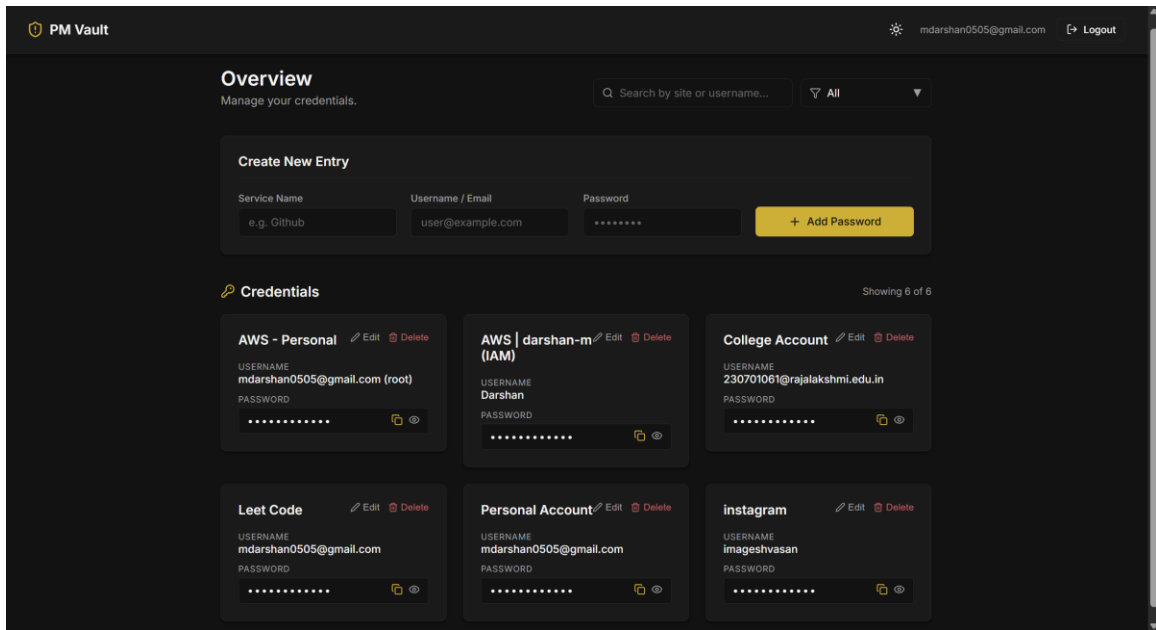


Figure 16: Project Screenshot

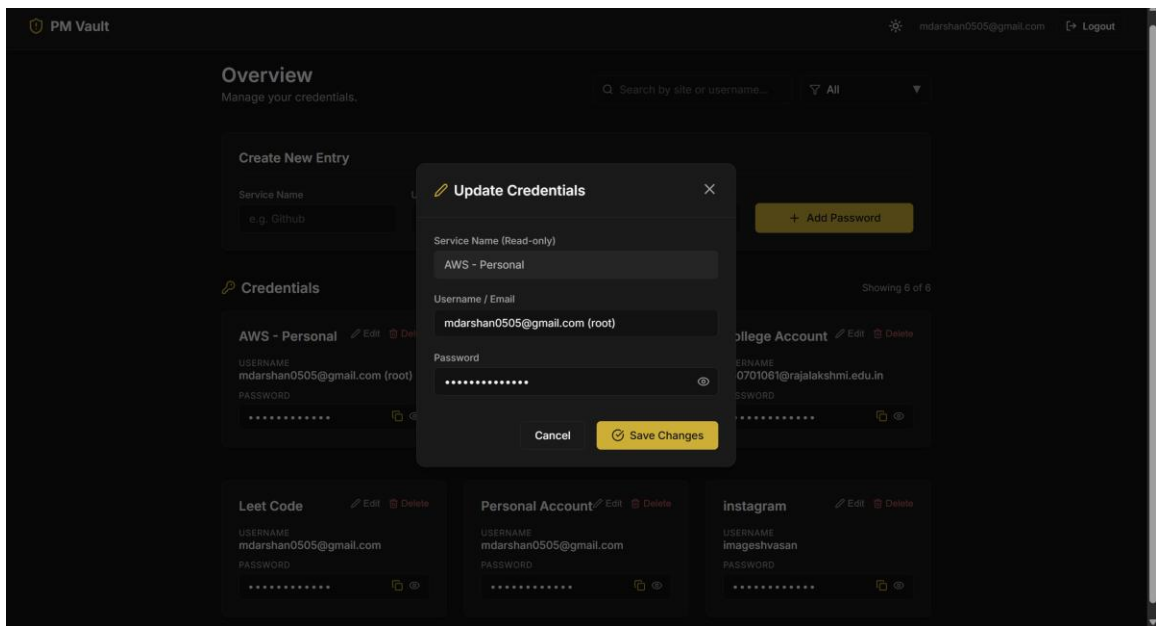


Figure 17: Project Screenshot